

LAB 8

Diamond & Virtual Inheritance

The Diamond Problem and How virtual Solves It

Learning Objectives

By the end of this lab, you should be able to:

- Explain what diamond inheritance is and recognize it in a class diagram.
- Identify the ambiguity problem that diamond inheritance creates.
- Observe duplicate base-class instances in memory and the compiler errors they cause.
- Trace constructor execution order through a diamond hierarchy.
- Apply the virtual keyword to make a base class shared instead of duplicated.
- Explain why the most-derived class is responsible for initializing a virtual base.

1. What Is Diamond Inheritance?

Diamond inheritance happens when a class inherits from two classes that both derive from the same base. The class diagram literally looks like a diamond — one base at the top, two siblings in the middle, one descendant at the bottom.



The diamond shape — $A \rightarrow B, C \rightarrow D$



In this diagram:

- **Class A** — the base class at the top.
- **Classes B and C** — both derived from A (two independent children of A).
- **Class D** — derived from BOTH B and C (the most-derived class at the bottom).

This shape looks harmless. It is not. Without a special fix, an object of type D will end up holding two complete copies of A inside it — one inherited through B, another through C. That duplication is the source of every problem in this lab.

 Why this is a real problem

Two copies of A means two copies of every field A defines. If A has an int x, then D has B::A::x AND C::A::x — two different variables, both called x. When you write obj.x, the compiler cannot decide which one you mean, and your program will not compile.

2. Code 1 — Watching the Problem Happen

Goal: write the exact diamond from the diagram, then try to use it. The class declarations themselves compile — but the moment we try to access A's field, the compiler raises an ambiguity error.

The Code

```
#include <iostream>
using namespace std;

class A {
public:
    int x;
    A() {
        x = 0;
        cout << "Constructor A called" << endl;
    }
};

class B : public A {
public:
    B() {
        cout << "Constructor B called" << endl;
    }
};

class C : public A {
public:
    C() {
        cout << "Constructor C called" << endl;
    }
};

class D : public B, public C {
public:
    D() {
        cout << "Constructor D called" << endl;
    }
};

int main() {
    D obj;
    obj.x = 10;          // <-- THIS LINE will not compile
    return 0;
}
```

What Goes Wrong

Object `obj` of type `D` contains TWO complete copies of `A` — one came through `B`, the other through `C`. So `obj` has two fields both literally named `x`:

- `obj.B::A::x` — the `x` that lives inside the `B` sub-object.
- `obj.C::A::x` — the `x` that lives inside the `C` sub-object.

When you write `obj.x = 10`, the compiler has no way to choose between them and produces an ambiguity error.

✗ Compiler Error

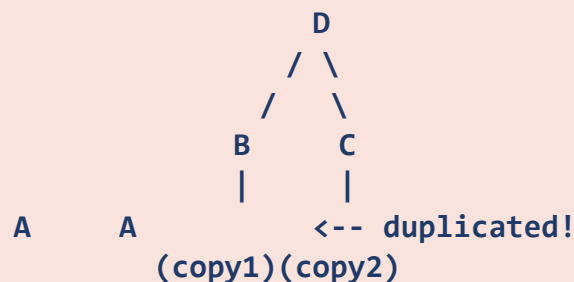
```
error: request for member 'x' is ambiguous
note: candidates are: int A::x [via B]
                    int A::x [via C]
```

📖 You CAN still access `x` — but it is painful

Writing `obj.B::x = 10;` and `obj.C::x = 10;` compiles, because you have told the compiler exactly which copy you mean. But now you have to choose every time, and you still have TWO independent `xs` — the memory is duplicated. That is not what we want for a single conceptual `A`.

How the Memory Looks

✗ Without virtual — `D` contains TWO copies of `A`



3. The Solution — Virtual Inheritance

C++ gives us a keyword to fix this: `virtual`. When B and C inherit from A using `virtual public A`, C++ ensures the entire hierarchy shares exactly ONE copy of A — no matter how many paths lead to it.

Syntax

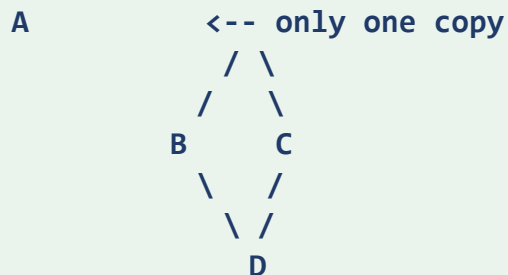
```
class B : virtual public A { /* ... */ };
class C : virtual public A { /* ... */ };
class D : public B, public C { /* ... */ };
```

What virtual Changes

- **A becomes a shared base.** B and C now refer to the SAME A sub-object, not their own private copies.
- **Only one instance of A exists** inside D — the duplication is gone.
- **Ambiguity disappears.** `obj.x` now refers to the one and only `A::x`.
- **Memory usage drops** because A's fields are stored once, not twice.

The New Memory Layout

✓ With virtual — ONE shared A



The new responsibility — initializing the shared base

There is one twist. When A is a virtual base, the responsibility for calling A's constructor moves all the way down to the MOST DERIVED class (here, D). Any `A(...)` calls inside B or C are silently IGNORED when an object of type D is being built. The next code example shows this in action — and it is the most surprising part of this lab.

4. Code 2 — Virtual Inheritance in Action

Goal: take the broken diamond and fix it with virtual. We also rig B, C, and D to each call A with a different value so we can prove which one actually wins.

The Code

```
#include <iostream>
using namespace std;

class A {
public:
    int x;
    A(int val) {
        x = val;
        cout << "Constructor A called with value: " << x << endl;
    }
};

class B : virtual public A {
public:
    B() : A(10) {                // <-- tries to set A(10)
        cout << "Constructor B called" << endl;
    }
};

class C : virtual public A {
public:
    C() : A(100) {              // <-- tries to set A(100)
        cout << "Constructor C called" << endl;
    }
};

class D : public B, public C {
public:
    D() : A(1000) {             // <-- D's choice wins
        cout << "Constructor D called" << endl;
    }
};

int main() {
    D obj;
    cout << "Value of x: " << obj.x << endl;
    return 0;
}
```

Code Breakdown

Code	What it does
<code>class B : virtual public A</code>	← B inherits from A virtually — meaning B agrees to share its A with anyone else who needs one.
<code>class C : virtual public A</code>	← C does the same. Now B and C will share a single A whenever they appear together in a derived class.
<code>class D : public B, public C</code>	← D inherits from B and C. Because A is virtual in both, D ends up with ONE A — not two.
<code>D() : A(1000)</code>	← D directly initializes A, even though A is not D's direct parent. This is allowed (and required) for virtual bases.
<code>B() : A(10), C() : A(100)</code>	← These calls are IGNORED when constructing an object of type D. D's call wins.
<code>obj.x</code>	← Compiles cleanly now — there is only one x to refer to.

Expected Output

► Output

```

Constructor A called with value: 1000
Constructor B called
Constructor C called
Constructor D called
Value of x: 1000

```



The proof — x is 1000, not 10 or 100

Look closely at the output: *Constructor A called with value: 1000*. B asked for 10, C asked for 100, but only D's request was honored. This is concrete proof that the most-derived class controls initialization of a virtual base.



What happens if D forgets to initialize A?

If you remove `: A(1000)` from D's constructor, the compiler will try to call A's default constructor. Since A in this example has no default constructor, you get a compile error.

Rule: whenever a virtual base lacks a default constructor, every concrete derived class MUST initialize it explicitly.



But B and C still work if used alone

If you instantiate just a B (not as part of a D), then B IS the most-derived class — and its `A(10)` will be honored. The "most-derived wins" rule only kicks in when something more derived exists. B and C are still valid standalone classes.

5. Code 3 — A Real-World Diamond (Professor)

Goal: a meaningful example. A Professor is both a Teacher and a Researcher. Both Teacher and Researcher ARE a Person. Without virtual inheritance, every Professor would carry two Person sub-objects — two names, two ages, total nonsense. Virtual inheritance fixes it.

The Class Hierarchy

✓ Person → Teacher, Researcher → Professor



The Code

```

#include <iostream>
using namespace std;

class Person {
protected:
    string name;
    int    age;
public:
    Person(string n = "", int a = 0) {
        name = n;
        age  = a;
        cout << "Person constructor called\n";
    }
    void showPersonInfo() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

class Teacher : virtual public Person {
protected:
    int employeeID;
public:
    Teacher(string n = "", int a = 0, int id = 0) : Person(n, a) {
        employeeID = id;
        cout << "Teacher constructor called\n";
    }
    void showTeacher() {

```

```

        cout << "Employee ID: " << employeeID << endl;
    }
};

class Researcher : virtual public Person {
protected:
    string researchArea;
public:
    Researcher(string n = "", int a = 0, string area = "") : Person(n, a) {
        researchArea = area;
        cout << "Researcher constructor called\n";
    }
    void showResearch() {
        cout << "Research Area: " << researchArea << endl;
    }
};

class Professor : public Teacher, public Researcher {
public:
    Professor(string n, int a, int eid, string area)
        : Person(n, a), // most-derived initializes virtual
base
        Teacher(n, a, eid),
        Researcher(n, a, area) {
        cout << "Professor constructor called\n";
    }
    void showProfessorInfo() {
        showPersonInfo();
        showTeacher();
        showResearch();
    }
};

int main() {
    Professor prof("Dr. Smith", 50, 9001, "AI");
    prof.showProfessorInfo();
    return 0;
}

```

Code Breakdown

Code	What it does
class Teacher : virtual public Person	← Teacher virtually inherits Person — agrees to share its Person.
class Researcher : virtual public Person	← Researcher does the same — so any class deriving from both will get ONE Person.

Code	What it does
<code>class Professor : public Teacher, public Researcher</code>	← Professor IS BOTH. Thanks to virtual, it has exactly one Person sub-object.
<code>: Person(n, a)</code>	← Professor — the most derived — initializes the shared Person directly.
<code>: Teacher(n, a, eid), Researcher(n, a, area)</code>	← Still chain to Teacher and Researcher so their OWN fields (employeeID, researchArea) get set.
<code>showProfessorInfo()</code>	← Calls all three show functions — name/age print exactly once because there is exactly one Person.

Expected Output

► Output

```
Person constructor called
Teacher constructor called
Researcher constructor called
Professor constructor called
Name: Dr. Smith, Age: 50
Employee ID: 9001
Research Area: AI
```



Watch the construction order carefully

Person runs FIRST — virtual bases are always constructed before any non-virtual bases, no matter where they appear in the inheritance chain. Then Teacher, then Researcher (in the order they appear in Professor's declaration), then finally Professor itself.



What the Teacher and Researcher Person calls are doing

Inside Teacher's constructor, *Person(n, a)* IS still there, but the compiler ignores it when building a Professor. It is only used when you build a standalone Teacher object (where Teacher itself is the most derived). The same applies to Researcher. This dual behavior is exactly why virtual inheritance is powerful.

6. Summary

Six things to take away from this lab:

- **Diamond inheritance** is the four-class $A \rightarrow B, C \rightarrow D$ pattern that creates duplicate base sub-objects.
- **Without virtual, the duplication causes ambiguity errors** the moment you try to access a base-class field directly.
- **virtual inheritance makes the base class shared** — one sub-object, not many. Use it on the immediate children of the shared base.
- **The most-derived class initializes the virtual base.** Any base-class calls inside intermediate classes are ignored when a more derived object is being built.
- **Virtual bases are constructed FIRST.** Before any non-virtual base, regardless of declaration order.
- **Use virtual only when you genuinely have a diamond.** It adds a tiny runtime cost (an extra pointer), so do not sprinkle it everywhere.

7. Practice Tasks

Task 1 — Reproduce the Bug, Then Fix It

Goal: feel the compile error with your own hands, then make it go away.

- Type Code 1 exactly as shown in Section 2 and try to compile it. Read the exact error message your compiler produces.
- Now add the virtual keyword in the right places (only B and C need it, NOT D) and try again. The error should disappear.
- Once it compiles, add `cout << obj.x;` right after `obj.x = 10;` and verify the output is 10.



Hint for Task 1

If A has no default constructor (i.e., it requires arguments), you will also need to add a `A(...)` call inside D's constructor. Otherwise A's default constructor (which provides `x = 0`) is fine.

Task 2 — Amphibious Vehicle (Diamond from Scratch)

Build a real-world diamond:

- **Base class:** Vehicle — has *brand* (string) and a function `showBrand()`.
- **Derived class:** LandVehicle — virtually inherits Vehicle, adds *wheels* (int).

- **Derived class:** WaterVehicle — virtually inherits Vehicle, adds *displacement* (double, in tons).
- **Most-derived class:** AmphibiousVehicle — inherits from BOTH LandVehicle and WaterVehicle.

Requirements

- Vehicle has a constructor that takes the brand. There should be NO default constructor.
- AmphibiousVehicle must initialize Vehicle, LandVehicle, and WaterVehicle explicitly in its constructor's initializer list.
- Provide a display function on AmphibiousVehicle that prints brand, wheels, and displacement — and verify brand prints exactly once.



Hint for Task 2

The signature should look something like: AmphibiousVehicle(string b, int w, double d) : Vehicle(b), LandVehicle(b, w), WaterVehicle(b, d) { ... }. Note that the brand value still gets passed to LandVehicle and WaterVehicle's constructors, but those calls to Vehicle inside them are IGNORED — only your direct Vehicle(b) call counts.

Task 3 — Teaching Assistant (Real-World Diamond)

Build a hierarchy modeling a university teaching assistant:

- **Base class:** Person — has *name* and *age*.
- **Derived class:** Student — virtually inherits Person, adds *studentID* and *cgpa*.
- **Derived class:** Employee — virtually inherits Person, adds *salary*.
- **Most-derived class:** TeachingAssistant — inherits from BOTH Student and Employee, adds *course* (the course they assist with).

Requirements

- Use constructors at every level. Have each constructor print a message so you can observe the order.
- TeachingAssistant must directly initialize Person.
- Provide a display function on TeachingAssistant that prints all five attributes (name, age, studentID, cgpa, salary, course) — and confirm name and age print exactly once.



What you should observe

When you run main and create one TeachingAssistant, the constructor messages should appear in this order: Person, Student, Employee, TeachingAssistant. If you see Person

printed TWICE, you forgot the virtual keyword somewhere — that is the diamond problem returning to haunt you.

⚠ Common mistake to avoid

Do NOT put virtual on TeachingAssistant's inheritance. Only Student and Employee (the classes right above the shared Person) need it. Adding virtual to TeachingAssistant's inheritance of Student and Employee would be meaningful only if some class derived further from TeachingAssistant — which there is not.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST